

Enhanced Traceroute and Network Diagnostics Toolkit
CSCE 4890 – Directed Study Final Report

Prepared by Lattamore Buckner Jr

University of North Texas

Department of Computer Science and Engineering

Supervised by Dr. Jacob Hochstetler

June 2026

1. Introduction	2
2. Background.....	3
3. Project Requirements.....	4
3.1 Overview	4
3.2 Functional.....	4
3.3 Non-Functional.....	5
4. Program Architecture	5
4.1 Overview	5
4.2 High-Level Structure.....	5
4.3 Major Components	6
4.4 Component Interaction	7
5. Implementation	7
5.1 Traceroute Module	7
5.2 Ping Module	8
5.3 TCP Latency Module	8
5.4 Analysis Module	9
5.4.1 Bottleneck Module	9
5.4.2 Route Comparison Module	10
5.4.3 Route Summary Module	10
5.5 JSON Export Module	11
5.6 Data Structures	11
5.6.1 Hop Structure.....	11
5.6.2 Fault Structure	12
5.6.3 PingResult Structure	12
5.6.4 TCPResult Structure	12
6. Testing and Results.....	12
6.1 Traceroute Results.....	13
6.2 Ping Results.....	16
6.3 TCP Results.....	16
6.4 Fault Injection Testing	17
6.5 Route Comparison Results	19

7. Challenges Encountered	22
7.1 Probe and Hop Data Model	22
7.2 Probe Analysis Logic	22
7.3 Handling Partial Network Data	23
7.4 Software Architecture Refactoring.....	24
8. Lessons Learned	24
8.1 From theory to real-world networking / Real-world ICMP limitations	24
8.2 Low-level networking challenges	25
8.3 Network performance is multi-dimensional.....	25
9. Future Improvements.....	26
9.1 Long-Term Network Analysis	26
9.2 Enhanced Reporting Capabilities	26
9.3 IPv6 Support.....	26
10. Conclusion	27
References	28

1. Introduction

The purpose of this project was to develop an enhanced traceroute and network analysis tool using the Go programming language. It was developed as part of a directed study to gain hands-on experience with low-level networking concepts, systems programming, and network analysis techniques commonly encountered in network engineering and cybersecurity. The project is a command-line network diagnostic toolkit that includes DNS resolution, TCP latency testing, ICMP ping operations, traceroute path discovery, hop-by-hop route analysis, network bottleneck detection, fault injection and resilience testing, route comparison, and JSON data export.

Through the development of this project I gained practical experience in areas such as network troubleshooting, raw socket programming, network performance analysis, and systems programming. Routing is one of the most critical concepts in networking, so understanding how network traffic flows across the internet is essential to understanding networks.

2. Background

Before I began developing the program for my network analysis tools, I studied the core concepts behind traceroute. Through various articles I became familiar with how traditional traceroute implementations typically use a packet's TTL (Time to Live) along with ICMP echo requests and replies to map out the path a packet takes from sender to recipient. To ensure a packet isn't sent from router to router indefinitely every packet stores a numerical value that is decremented each time the packet passes through a router. It subtracts one from the TTL count and then passes it to the next location in the network. If the TTL count is equal to zero after the subtraction, the router discards the packet and then sends an ICMP message back to the originating host. This process repeats with progressively larger TTL values until the packet arrives at its intended destination. The ICMP (The Internet Control Message Protocol) is primarily used for error reporting and diagnostics. It is commonly associated with utilities such

as ping “a basic Internet command used in computer network administration to test and verify the availability of a destination IP address” and traceroute because they rely on it to test connectivity and trace the route of packets. ICMP based tools are good for verifying connectivity and identifying the route packets are taking, but they fail to provide a complete picture of application performance. In some cases, a host may respond to ICMP requests while still experiencing delays or failures in application traffic. TCP latency testing measures the time required to establish a connection between a client and a target host. This differs from ICMP ping which measures the RTT (Round-Trip-Time) of ICMP echo messages. This allows the responsiveness of services on specific ports to be tested. This provides a more accurate representation of the performance users are experiencing at the network application level. Continuously monitoring, evaluating, and optimizing network infrastructure is essential and has several benefits including cost reduction and improved end-user experience.

3. Project Requirements

3.1 Overview

This section outlines the functional and non-functional requirements established for the network analysis toolkit. The functional requirements define the core features and capabilities the application is expected to provide, while the non-functional requirements describe the design constraints and quality attributes that guided the development process. Together, these requirements establish the objectives and standards the project must satisfy.

3.2 Functional

Perform Traceroute Operations: When given a valid host, the application should identify and display the sequence of network hops between the source and destination host.

Measure RTT: Round-trip time should be measured to estimate network latency.

Calculate Packet Loss: The tool should identify missing and unanswered probes and return a percentage.

Calculate Jitter: The toolkit should calculate the variation between the average latency of each hop.

Measure TCP Latency: The toolkit should measure the time it takes to establish a TCP connection to a specific host and port.

Compare Routes: The application should allow two traceroute results to be compared, identifying route changes as well as significant differences in RTT, packet loss, jitter, and bottleneck scores.

Export Results to JSON: Data should be collected and stored for long-term analysis.

Analyze the full route and each hop within the route calculating packet loss, RTT, jitter, and latency across all sent probes.

3.3 Non-Functional

Written in Go: Go was selected because it offers an easy-to-read syntax, with the advantages of statically typed languages. Also, Go is efficient in terms of speed and executable size because it is a compiled language.

Cross-platform compatibility: The application should be able to run on multiple operating systems

Modular Code Structure: The program should be organized into reusable and maintainable components.

Command-Line Interface: Users should be able to interact with the application through terminal commands and prompts.

4. Program Architecture

4.1 Overview

This section provides a high-level overview of the network analysis toolkit and its modular architecture. It describes the project's directory and file structure, allowing readers to efficiently navigate the codebase, locate relevant files, and understand how the major components are

organized and interact with one another. The toolkit is organized into modules responsible for user interaction, network probing, route analysis, and data export.

4.2 High-Level Structure

The `main.go` serves as the entry point for the application. It handles user interaction, displays the command-line interface, and coordinates the execution of the networking modules. The system follows a layered processing pipeline.

Raw network data is collected with the data collection layer which includes the traceroute, ping, and TCP latency modules. Here performance metrics such as RTT, jitter, and packet loss are gathered.

The data is then processed by the analysis layer. Here, collected data including average RTT, jitter, packet loss, and RTT increases between hops are calculated. This is also where bottleneck detection takes place using a scoring-based system that identifies potential congestion points along a network path.

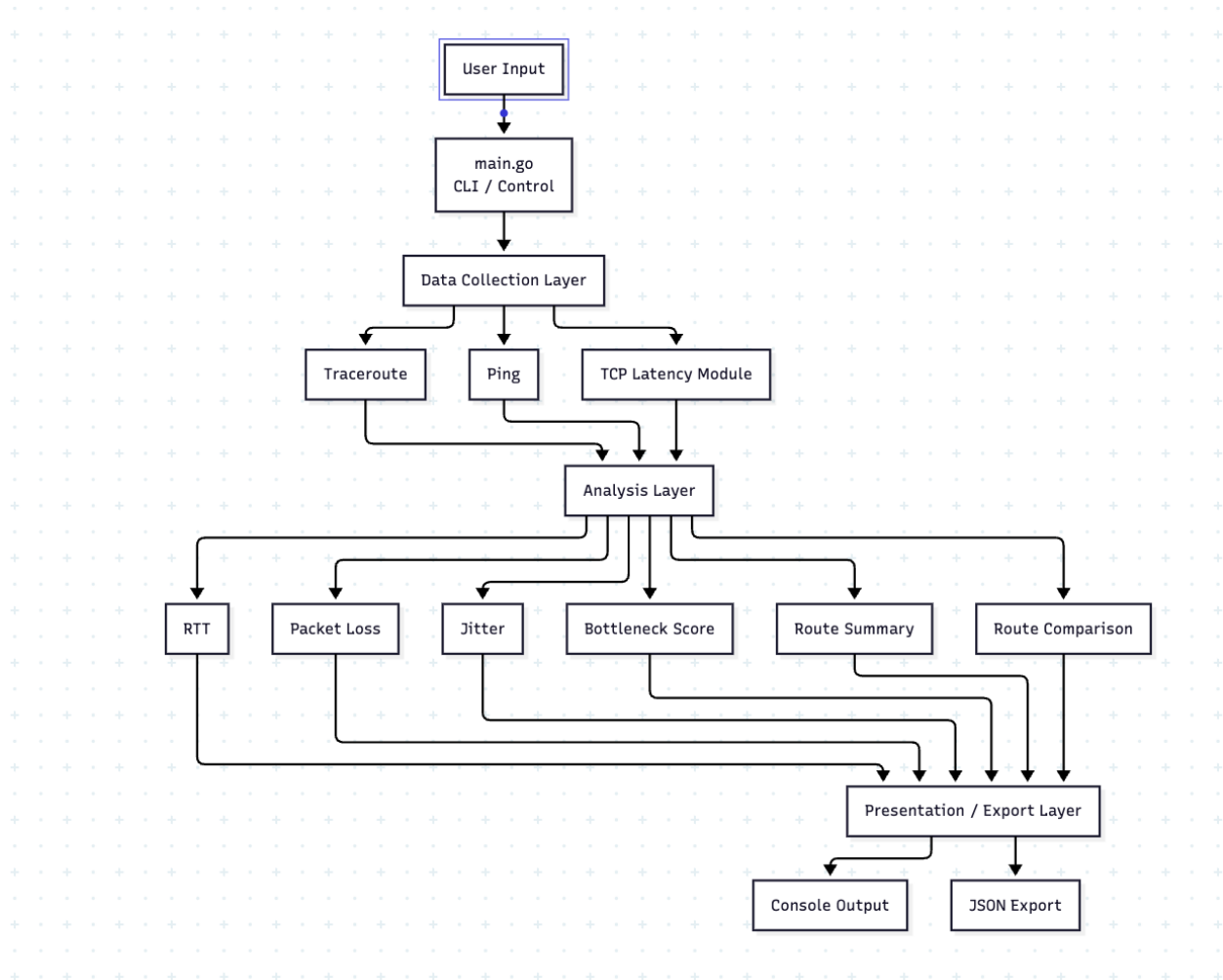
The last layer is the presentation and export layer which displays the analyzed data through the command-line interface and supports the option to export traceroute, ping, and TCP latency to JSON files for long-term analysis.

4.3 Major Components

- `main.go` – Entry point and menu/CLI handling.
- `traceroute.go` – Implements the traceroute utility, performs DNS resolution, transmits ICMP probes, and collects hop-by-hop network measurements.
- `pingTool.go` – Executes ICMP echo requests and measures RTT.
- `tcpLat.go` – Measures TCP connection establishment time.

- analyzeHop.go – Processes collected probe data by calculating RTT statistics, packet loss, jitter, bottleneck scores, route summaries, and route comparisons.
- exportJSON.go – Serializes collected data to JSON.

4.4 Component Interaction



5. Implementation

5.1 Traceroute Module

The purpose of the traceroute module is to discover the network path to a destination host. This is accomplished by first resolving the target hostname to an IPv4 address. Next, a raw ICMP packet listener is opened on all interfaces and an ICMP echo request packet is transmitted with

incrementally increasing Time-to-Live (TTL) values. Three probes are sent for each TTL value to improve measurement reliability, and either an ICMP Time Exceeded message from intermediate routers or an Echo reply message from the destination host is recorded by the module. In addition, the module includes a feature for fault injection. Whenever enabled artificial delay and packet loss are introduced into the network path. This simulates network degradation and allows for controlled test scenarios when running route comparison and bottleneck analysis.

5.2 Ping Module

The ping module performs repeated ICMP Echo Requests to measure host latency and network stability. A total of three probes are sent to the target host. The timer for RTT starts immediately before the packet is transmitted and ends once the corresponding ICMP reply packet is received. Once a successful probe is received, the minimum and maximum RTT values are updated. The average RTT is calculated by summing all successful RTT measurements and dividing by the number of successful responses. To calculate jitter, first the absolute difference between consecutive probes is calculated. The differences are then added up and divided by the number of intervals (successful probes minus one) to obtain the average variation in latency. Packet loss is calculated by subtracting the number of successful probes from the total number of probes (three), dividing by three, and then multiplying by 100 to produce a percentage.

5.3 TCP Latency Module

The TCP latency module measures application-level connectivity by evaluating DNS resolution and TCP connection establishment time. The hostname is first resolved into one or more IP addresses, and the time required for this process is recorded as the DNS lookup time. After DNS resolution is complete, a connection timer is started. The hostname is combined with the HTTPS

port into a valid network address, and an attempt is made to establish a TCP connection. After each attempt, the elapsed connection time is recorded. This process is repeated three times, with the minimum and maximum connection times being updated after each successful attempt. To compute the average connection time, the elapsed time from successful connections is summed and then divided by the number of successful connections. Packet loss is calculated by subtracting the number of successful connection attempts from the total number of connection attempts (three), dividing by three, and then multiplying by 100 to produce a percentage.

5.4 Analysis Module

The analysis module processes raw probe data collected by network measurement tools and computes performance metrics used throughout the toolkit. Packet loss percentage is calculated by subtracting the total number of successful probes from the total number of sent probes, dividing this number by the total number of sent probes, and multiplying by 100 to produce a percentage. The average RTT is calculated by iterating through the RTT measurements for each collected hop, summing all successful RTT measurements and dividing by the number of successful responses. Minimum and maximum RTT values are also updated during this process. Jitter is calculated by first computing the absolute difference between consecutive RTT measurements. These differences are summed and then divided by the number of intervals (successful probes minus one) to obtain the average variation in latency. These metrics are stored within each Hop structure for bottleneck detection and reporting modules.

5.4.1 Bottleneck Module

The bottleneck detection module identifies points of congestion or performance degradation along the network route. Each hop apart from the first hop is compared against the previous hop and is assigned a bottleneck score based on several performance metrics. If RTT increased by

over 30 ms compared to the previous hop, two points are added to the score. If packet loss exceeds 20%, two points are added. Unless packet loss is 100% (99.9% or greater), then four points are added. If jitter exceeds 30 ms then one point is added. If the average RTT is over 100 ms then two points are added. If the maximum RTT is over 200 ms then one point is added. Each condition contributes to a cumulative bottleneck score which is then mapped to severity levels. If no points are accumulated, then severity corresponds to none. If one point is accumulated, then the severity corresponds to Minor. If two or three points are accumulated, then the severity corresponds to Moderate. If four or more points are accumulated, then severity corresponds to Severe. This scoring system provides a simple method for identifying and classifying potential network bottlenecks.

5.4.2 Route Comparison Module

The route comparison compares two traceroute results to identify changes in network performance over time. A baseline route is compared against a second route with the option to enable fault injection for the second. This allows for changes caused by network congestion, route adjustment, or fault injection to be identified. The module first checks if the routes contain the same number of hops; different hop values could indicate route changes. Each hop is then compared using corresponding TTL values. If the IP addresses differ, a route change has occurred. For hops following the same path the module compares packet loss, average RTT, jitter, and bottleneck scores. Using predefined thresholds significant changes are reported. Packet losses that change by more than 20% are reported as significant. RTT and jitter changes greater than 30 ms are reported. Bottleneck scores are reported to highlight network improvements or declines.

5.4.3 Route Summary Module

The route summary module provides overall route health metrics. The total number of hops in a network path is reported, giving a basic measure of network length. The average RTT is calculated and displayed by aggregating the average RTT values from each hop and dividing by the total number of valid responses. This gives a general indication of network latency. The average packet loss is calculated and displayed by summing and dividing the total number of loss probes by the total number of probes sent, producing an overall route percentage. Additionally, the hop with the highest bottleneck score is identified and displayed, highlighting the point in the path with the most significant performance degradation.

5.5 JSON Export Module

The JSON export module serializes and saves traceroute, ping, and TCP latency results into a structured JSON format. This module converts Hop data and associated network metrics into machine-readable representation. This includes metrics such as RTT, packet loss, jitter, and bottleneck scores. This allows results to be archived, shared, or imported into other analysis tools for long term analysis.

5.6 Data Structures

5.6.1 Hop Structure

There were four data structures created to store raw measurements and computed analysis metrics. The Hop structure represents the core unit of traceroute analysis and contains the TTL value for each hop along with the hostname and IP address of the responding router. It also stores RTT measurements collected from multiple probes, including the average RTT across successful probes, as well as the minimum and maximum RTT values discovered in the hop. Additionally, it tracks the total number of probes that were sent and the total number of probes that received a

reply which is used for calculating the percentage of probes that were lost. The variation between RTT measurements (jitter), RTT increases relative to the previous hop, and the cumulative bottleneck score used to identify bottlenecks are also stored in the structure. Finally, the structure includes a severity classification as well as multiple Boolean flags to identify performance issues such as RTT increase, high packet loss, high jitter, high average RTT, and high max RTT.

5.6.2 Fault Structure

The Fault structure encapsulates packet loss and delay simulation settings used for fault injection during testing. This structure includes an Enabled bool value that enables or disables fault injection. A drop rate value for the probability of simulated packet loss. Along with a delay variable that defines the maximum artificial delay applied to a probe.

5.6.3 PingResult Structure

The PingResult structure stores measurements from ICMP echo requests and computed statistics. Included in the structure is the hostname of the responding router. The average RTT across successful probes. The minimum and maximum RTT value. Jitter (RTT variation). As well as the percentage of probes that were lost.

5.6.4 TCPResult Structure

The TCPResult structure stores results from TCP-based latency. Included in the structure is the name of the target host. The time required to resolve a hostname to the IP address. The average TCP connection latency across successful attempts. The fastest and slowest successful TCP connection times. The number of successful connections established and the percentage of failed connection attempts.

6. Testing and Results

6.1 Traceroute Results

```
Hop 23 Probe 1
Request timed out.

Hop 23 Probe 2
Request timed out.

Hop 23 Probe 3
Request timed out.

Hop 24 Probe 1
Request timed out.

Hop 24 Probe 2
Request timed out.

Hop 24 Probe 3
Request timed out.

Hop 25 Probe 1
Request timed out.

Hop 25 Probe 2
Request timed out.

Hop 25 Probe 3
Request timed out.

Hop 26 Probe 1
Request timed out.

Hop 26 Probe 2
Request timed out.

Hop 26 Probe 3
Request timed out.

Hop 27 Probe 1
RTT: 1.385536458s
Received reply from 127.0.0.1
Received ICMP message of type destination unreachable

Hop 27 Probe 2
Request timed out.

Hop 27 Probe 3
Request timed out.

Hop 28 Probe 1
Request timed out.

Hop 28 Probe 2
Request timed out.

Hop 28 Probe 3
Request timed out.

Hop 29 Probe 1
RTT: 39.595959ms
Received reply from 142.250.113.100
Destination reached!
Echo Reply ID: 13024, Seq: 290, Data: WADDUP
TTL: 29

Hop 29 Probe 2
RTT: 122.461875ms
Received reply from 142.250.113.100
Destination reached!
Echo Reply ID: 13024, Seq: 291, Data: WADDUP
TTL: 29

Hop 29 Probe 3
RTT: 27.172708ms
Received reply from 142.250.113.100
Destination reached!
Echo Reply ID: 13024, Seq: 292, Data: WADDUP
TTL: 29

Traceroute complete.
```

```

o dabuckners@Mac traceroute-project % sudo go run .
Password:
Enter a host: google.com

Current Host: google.com
1. Run Traceroute
2. Run Traceroute(Optional Fault Injection)
3. Hop Analytics
4. Export Route A JSON
5. Export Route B JSON
6. Change Host
7. Run Ping
8. Run TCP Latency Tool
9. Export Ping JSON
10. Export TCP JSON
11. Exit

Select an option:
1

Hop 1 Probe 1
RTT: 4.300584ms
Received reply from 192.168.12.1
Received Time Exceeded message: TTL: 1| Reply From tmo-g5ar.lan| Time Elapsed: 4.300584ms|

Hop 1 Probe 2
RTT: 6.087333ms
Received reply from 192.168.12.1
Received Time Exceeded message: TTL: 1| Reply From tmo-g5ar.lan| Time Elapsed: 6.087333ms|

Hop 1 Probe 3
RTT: 4.832791ms
Received reply from 192.168.12.1
Received Time Exceeded message: TTL: 1| Reply From tmo-g5ar.lan| Time Elapsed: 4.832791ms|

Hop 2 Probe 1
RTT: 4.701917ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 2| Reply From 192.0.0.1| Time Elapsed: 4.701917ms|

Hop 2 Probe 2
RTT: 4.305542ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 2| Reply From 192.0.0.1| Time Elapsed: 4.305542ms|

Hop 2 Probe 3
RTT: 4.848958ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 2| Reply From 192.0.0.1| Time Elapsed: 4.848958ms|

Hop 3 Probe 1
RTT: 57.822291ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 3| Reply From 192.0.0.1| Time Elapsed: 57.822291ms|

Hop 3 Probe 2
RTT: 48.313084ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 3| Reply From 192.0.0.1| Time Elapsed: 48.313084ms|

Hop 3 Probe 3
RTT: 47.564458ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 3| Reply From 192.0.0.1| Time Elapsed: 47.564458ms|

Hop 4 Probe 1
Request timed out.

Hop 4 Probe 2
Request timed out.

Hop 4 Probe 3
Request timed out.

Hop 5 Probe 1
RTT: 34.773042ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 5| Reply From 192.0.0.1| Time Elapsed: 34.773042ms|

Hop 5 Probe 2
RTT: 39.596667ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 5| Reply From 192.0.0.1| Time Elapsed: 39.596667ms|

Hop 5 Probe 3
Request timed out.

```


The traceroute results identified the network path to the target host. Several of the intermediate hops did not respond, which is expected due to ICMP filtering.

6.2 Ping Results

Ping Results:	
Host	lcdfwna-in-f100.1e100.net
Average RTT	41.571111ms
Maximum RTT	74.546084ms
Minimum RTT	16.620459ms
Jitter	28.962812ms
Packet Loss	0.00%

The Ping module was tested against Google using three ICMP echo requests probes to measure stability and latency. The target host resolved to lcdfwna-in-f100.1e100.net, which is part of Google network's infrastructure. The results show an average RTT of 41.57, a maximum RTT of 74.55, a minimum RTT of 16.62, Jitter was measured at 28.96, and all three probes successfully reached the destination. The results indicate a stable connection and acceptable latency.

6.3 TCP Results

TCP Results:	
Host	google.com
DNS Lookup Time	6.948333ms
Average Connection Time	31.650653ms
Maximum Connection Time	37.898792ms
Minimum Connection Time	27.085167ms
Packet Loss	0.00%

The TCP latency module established TCP connections to the target host over HTTPS (port 443). This test measures DNS resolution time as well as overall connection latency and stability. Three connection attempts evaluate the consistency in connection performance. The DNS lookup time completed in 6.95ms, the average connection time was 31.65ms, the maximum connection time

was 37.9ms, the minimum connection time was 27.1ms, and all three probes successfully reached the destination. These results indicate strong application-level connectivity, fast DNS resolution time, and a reliable connection.

6.4 Fault Injection Testing

```
Current Host: google.com
1. Run Traceroute
2. Run Traceroute(Optional Fault Injection)
3. Hop Analytics
4. Export Route A JSON
5. Export Route B JSON
6. Change Host
7. Run Ping
8. Run TCP Latency Tool
9. Export Ping JSON
10. Export TCP JSON
11. Exit

Select an option:
2
Enable fault injection? (Y/N): y
Fault injection enabled.

Simulating delay of 92.238351ms for Hop 1 Probe 1

Hop 1 Probe 1
RTT: 97.185042ms
Received reply from 192.168.12.1
Received Time Exceeded message: TTL: 1| Reply From tmo-g5ar.lan| Time Elapsed: 97.185042ms|

Simulating delay of 28.727703ms for Hop 1 Probe 2

Hop 1 Probe 2
RTT: 32.500291ms
Received reply from 192.168.12.1
Received Time Exceeded message: TTL: 1| Reply From tmo-g5ar.lan| Time Elapsed: 32.500291ms|

Simulating delay of 70.399026ms for Hop 1 Probe 3

Hop 1 Probe 3
RTT: 74.618542ms
Received reply from 192.168.12.1
Received Time Exceeded message: TTL: 1| Reply From tmo-g5ar.lan| Time Elapsed: 74.618542ms|

Simulating delay of 8.719014ms for Hop 2 Probe 1

Hop 2 Probe 1
RTT: 12.879125ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 2| Reply From 192.0.0.1| Time Elapsed: 12.879125ms|

Simulating delay of 88.827517ms for Hop 2 Probe 2

Hop 2 Probe 2
RTT: 97.690792ms
Received reply from 192.0.0.1
Received Time Exceeded message: TTL: 2| Reply From 192.0.0.1| Time Elapsed: 97.690792ms|

Hop 2 Probe 3
Simulating packet loss.
Request timed out.

Hop 3 Probe 1
Request timed out.

Hop 3 Probe 2
Request timed out.

Hop 3 Probe 3
Simulating packet loss.
Request timed out.

Hop 4 Probe 1
Request timed out.

Hop 4 Probe 2
Request timed out.

Hop 4 Probe 3
Request timed out.

Hop 5 Probe 1
Request timed out.

Hop 5 Probe 2
Request timed out.

Hop 5 Probe 3
Request timed out.
```

```
Hop 26 Probe 2
Request timed out.

Hop 26 Probe 3
Request timed out.

Hop 27 Probe 1
Request timed out.

Hop 27 Probe 2
Request timed out.

Hop 27 Probe 3
Request timed out.

Hop 28 Probe 1
Request timed out.

Hop 28 Probe 2
Request timed out.

Hop 28 Probe 3
Request timed out.

Hop 29 Probe 1
Request timed out.

Hop 29 Probe 2
Request timed out.

Hop 29 Probe 3
Request timed out.

Hop 30 Probe 1
Request timed out.

Hop 30 Probe 2
Request timed out.

Hop 30 Probe 3
Request timed out.

Hop 31 Probe 1
Request timed out.

Hop 31 Probe 2
Request timed out.

Hop 31 Probe 3
Request timed out.

Simulating delay of 31.473784ms for Hop 32 Probe 1

Hop 32 Probe 1
RTT: 99.861458ms
Received reply from 142.250.115.100
Destination reached!
Echo Reply ID: 13024, Seq: 320, Data: WADDUP
TTL: 32

Simulating delay of 98.578966ms for Hop 32 Probe 2

Hop 32 Probe 2
RTT: 136.808959ms
Received reply from 142.250.115.100
Destination reached!
Echo Reply ID: 13024, Seq: 321, Data: WADDUP
TTL: 32

Simulating delay of 42.140624ms for Hop 32 Probe 3

Hop 32 Probe 3
RTT: 120.20875ms
Received reply from 142.250.115.100
Destination reached!
Echo Reply ID: 13024, Seq: 322, Data: WADDUP
TTL: 32

Traceroute complete.
```

6.5 Route Comparison Results

```
Route Comparison Analysis:

Route length changed: Route 1 has 28 hops, Route 2 has 30 hops
Significant performance change detected at Hop 1 (192.168.12.1)
- RTT increased by 77.005257ms
- Packet Loss increased by 33.33%
- Jitter increased by 41.680959ms
- Bottleneck Score increased by 3

Comparing Hop 1:
Host: tmo-g5ar.lan | IP: 192.168.12.1
Avg RTT: Route 1: 5.456389ms -> Route 2: 82.461646ms
Packet Loss: Route 1: 0.00% -> Route 2: 33.33%
Jitter: Route 1: 3.311917ms -> Route 2: 44.992876ms
RTT Increase: Route 1: 0s -> Route 2: 0s
Max RTT: Route 1: 9.463167ms -> Route 2: 104.958084ms
Min RTT: Route 1: 2.839333ms -> Route 2: 59.965208ms
Bottleneck Score: Route 1: 0 -> Route 2: 3
Severity: Route 1: None -> Route 2: Moderate

-----

Significant performance change detected at Hop 2 (192.0.0.1)
- RTT increased by 33.322861ms
- Packet Loss increased by 33.33%
- Jitter increased by 43.884543ms
- Bottleneck Score increased by 3

Comparing Hop 2:
Host: 192.0.0.1 | IP: 192.0.0.1
Avg RTT: Route 1: 6.737139ms -> Route 2: 40.06ms
Packet Loss: Route 1: 0.00% -> Route 2: 33.33%
Jitter: Route 1: 2.512291ms -> Route 2: 46.396834ms
RTT Increase: Route 1: 1.28075ms -> Route 2: -42.401646ms
Max RTT: Route 1: 8.412ms -> Route 2: 63.258417ms
Min RTT: Route 1: 4.234958ms -> Route 2: 16.861583ms
Bottleneck Score: Route 1: 0 -> Route 2: 3
Severity: Route 1: None -> Route 2: Moderate

-----

Significant performance change detected at Hop 3 (192.0.0.1)
- Packet Loss increased by 33.33%
- Jitter decreased by 51.131938ms
- Bottleneck Score increased by 1

Comparing Hop 3:
Host: 192.0.0.1 | IP: 192.0.0.1
Avg RTT: Route 1: 50.755361ms -> Route 2: 72.080354ms
Packet Loss: Route 1: 0.00% -> Route 2: 33.33%
Jitter: Route 1: 75.556396ms -> Route 2: 24.424458ms
RTT Increase: Route 1: 44.018222ms -> Route 2: 32.020354ms
Max RTT: Route 1: 101.126292ms -> Route 2: 84.292583ms
Min RTT: Route 1: 13.698292ms -> Route 2: 59.868125ms
Bottleneck Score: Route 1: 3 -> Route 2: 4
Severity: Route 1: Moderate -> Route 2: Severe

-----

Comparing Hop 4:
Host: * | IP: *
Avg RTT: Route 1: 0s -> Route 2: 0s
Packet Loss: Route 1: 100.00% -> Route 2: 100.00%
Jitter: Route 1: 0s -> Route 2: 0s
RTT Increase: Route 1: -50.755361ms -> Route 2: -72.080354ms
Max RTT: Route 1: 0s -> Route 2: 0s
Min RTT: Route 1: 0s -> Route 2: 0s
Bottleneck Score: Route 1: 4 -> Route 2: 4
Severity: Route 1: Severe -> Route 2: Severe

-----

Significant performance change detected at Hop 5 (192.0.0.1)
- RTT increased by 30.975668ms
- Jitter increased by 61.372083ms

Comparing Hop 5:
Host: 192.0.0.1 | IP: 192.0.0.1
Avg RTT: Route 1: 46.13177ms -> Route 2: 77.107438ms
Packet Loss: Route 1: 33.33% -> Route 2: 33.33%
Jitter: Route 1: 42.979459ms -> Route 2: 104.351542ms
RTT Increase: Route 1: 46.13177ms -> Route 2: 77.107438ms
Max RTT: Route 1: 67.6215ms -> Route 2: 129.283209ms
Min RTT: Route 1: 24.642041ms -> Route 2: 24.931667ms
Bottleneck Score: Route 1: 3 -> Route 2: 3
Severity: Route 1: Moderate -> Route 2: Moderate
```

```

Comparing Hop 9:
Host: 10.180.165.253 | IP: 10.180.165.253
Avg RTT: Route 1: 29.568611ms -> Route 2: 72.115083ms
Packet Loss: Route 1: 0.00% -> Route 2: 66.67%
Jitter: Route 1: 3.139479ms -> Route 2: 0s
RTT Increase: Route 1: 37.486µs -> Route 2: -27.473708ms
Max RTT: Route 1: 33.201542ms -> Route 2: 72.115083ms
Min RTT: Route 1: 26.922583ms -> Route 2: 72.115083ms
Bottleneck Score: Route 1: 0 -> Route 2: 2
Severity: Route 1: None -> Route 2: Moderate

-----

Route change detected at Hop 10: 10.177.9.162 -> 10.177.99.128
Significant performance change detected at Hop 11 (10.164.178.239)
- Packet Loss decreased by 33.33%
- Jitter increased by 33.922251ms
- Bottleneck Score increased by 1

Comparing Hop 11:
Host: 10.164.178.239 | IP: 10.164.178.239
Avg RTT: Route 1: 29.182375ms -> Route 2: 57.929208ms
Packet Loss: Route 1: 66.67% -> Route 2: 33.33%
Jitter: Route 1: 0s -> Route 2: 33.922251ms
RTT Increase: Route 1: -8.106152ms -> Route 2: 27.174916ms
Max RTT: Route 1: 29.182375ms -> Route 2: 74.890334ms
Min RTT: Route 1: 29.182375ms -> Route 2: 40.968083ms
Bottleneck Score: Route 1: 2 -> Route 2: 3
Severity: Route 1: Moderate -> Route 2: Moderate

-----

Significant performance change detected at Hop 12 (173.194.122.144)
- RTT increased by 55.498666ms
- Packet Loss decreased by 66.67%
- Bottleneck Score decreased by 2

Comparing Hop 12:
Host: 173.194.122.144 | IP: 173.194.122.144
Avg RTT: Route 1: 31.957209ms -> Route 2: 87.455875ms
Packet Loss: Route 1: 66.67% -> Route 2: 0.00%
Jitter: Route 1: 0s -> Route 2: 24.752687ms
RTT Increase: Route 1: 2.774834ms -> Route 2: 29.526667ms
Max RTT: Route 1: 31.957209ms -> Route 2: 116.268583ms
Min RTT: Route 1: 31.957209ms -> Route 2: 66.763209ms
Bottleneck Score: Route 1: 2 -> Route 2: 0
Severity: Route 1: Moderate -> Route 2: None

-----

Route change detected at Hop 13: 192.178.73.245 -> 74.125.242.103
Route change detected at Hop 14: 172.253.77.150 -> 172.253.176.28
Route change detected at Hop 15: * -> 192.178.125.55

Comparing Hop 16:
Host: * | IP: *
Avg RTT: Route 1: 0s -> Route 2: 0s
Packet Loss: Route 1: 100.00% -> Route 2: 100.00%
Jitter: Route 1: 0s -> Route 2: 0s
RTT Increase: Route 1: 0s -> Route 2: -89.895729ms
Max RTT: Route 1: 0s -> Route 2: 0s
Min RTT: Route 1: 0s -> Route 2: 0s
Bottleneck Score: Route 1: 4 -> Route 2: 4
Severity: Route 1: Severe -> Route 2: Severe

-----

Route change detected at Hop 17: 192.178.255.140 -> 216.239.63.98
Route change detected at Hop 18: 216.239.63.153 -> 142.250.235.155

Comparing Hop 19:
Host: * | IP: *
Avg RTT: Route 1: 0s -> Route 2: 0s
Packet Loss: Route 1: 100.00% -> Route 2: 100.00%
Jitter: Route 1: 0s -> Route 2: 0s
RTT Increase: Route 1: -34.495805ms -> Route 2: -110.606521ms
Max RTT: Route 1: 0s -> Route 2: 0s
Min RTT: Route 1: 0s -> Route 2: 0s
Bottleneck Score: Route 1: 4 -> Route 2: 4
Severity: Route 1: Severe -> Route 2: Severe

-----

Comparing Hop 20:
Host: * | IP: *
Avg RTT: Route 1: 0s -> Route 2: 0s
Packet Loss: Route 1: 100.00% -> Route 2: 100.00%
Jitter: Route 1: 0s -> Route 2: 0s

```

The route comparison module compared a baseline traceroute with a second traceroute performed under simulated network conditions. The comparison detected an increase in route length from 28 hops to 30 hops along with multiple route changes. Significant performance degradation was also identified, including RTT increases of up to 77.01 ms, packet loss increases of 33.33%, and bottleneck score increases of up to 3 at several hops. The results demonstrate the module's ability to detect changes in both network topology and network performance while identifying potential bottlenecks along the route.

7. Challenges Encountered

7.1 Probe and Hop Data Model

There were several issues encountered during the planning, design, and implementation of these network analysis tools. One of the most significant challenges involved the data model design for probes and hops within traceroute functionality. For typical traceroute models, each hop contains multiple probes. During the early stages of development, some of my probes were sometimes treated like hops. This led to several downstream issues such as RTT values not being grouped together correctly, broken packet loss, jitter errors, and messy outputs. To resolve these errors, I altered lookup logic by checking for hops with identical TTLs before creating new Hop entries and appending RTT values to existing hops when TTLs do match. This ensures that all probes are stored with their associated hop.

7.2 Probe Analysis Logic

There were also significant issues with the probe analysis logic during development. Initially, the program lacked a consistent definition of probe success and failure states. This led to inconsistent assumptions regarding what counted as a successful probe, what counted as a valid

hop result, and when statistical calculations should be performed. This resulted in several analysis errors. RTT averages reset incorrectly; introducing the possibility of division by zero errors occurring during jitter calculation. This also caused packet loss to only count replies rather than all sent probes. False RTT increase detections when comparing hops was another issue. In addition, some calculations were performed without validating previous hop success, which resulted in misleading analysis. To resolve these issues, the probe analysis logic was redesigned to use clearer success and failure conditions for each probe. I moved total latency tracking variables outside the probe loop ensuring averages were correctly calculated across each probe, implemented timeout handling using `net.DialTimeout()`, and placed timeout handling inside the probe loop so each connection attempt has its own timeout. Additional validation checks were implemented to ensure RTT increase detection only occurred when previous hops had successful responses.

7.3 Handling Partial Network Data

During development there were also issues related to unsafe indexing caused by assumptions about complete probe data. The analysis logic initially assumed that each hop would contain a fixed number of RTT measurements. This failed to account for missing RTT values from probes that either timed out or were unresponsive. This occasionally produced index out of range errors due to my analysis attempting to access RTT values that did not exist. Partial responses are expected in real world network environments, so this was a major issue. To resolve these issues, hardcoded RTT indexing was replaced with iterative processing using: `for _, rtt := range h.RTTs`. This allowed RTT values to be processed dynamically. Additional validation checks using `len(h.RTTs)` were also added before calculations to ensure sufficient data existed. I also

refactored the analysis system to accept []Hop as input, allowing calculations to operate directly on structured hop data.

7.4 Software Architecture Refactoring

Architecture mixing also had to be addressed during development. Initially, data collection, analysis, and presentation logic were coupled throughout the program. Analysis code was embedded within testing code, presentation logic was placed inside analysis functions, and output formatting was tied closely into internal program logic. This made debugging more difficult, reduced function reusability, and limited scalability as additional features were added. To resolve these issues the program architecture was refactored to separate responsibilities more clearly. I moved calculations and presentation into their own distinct modules. AnalyzeHop() function for hop-level analysis. Reporting and presentation were separated into a PrintHopAnalysis() function.

8. Lessons Learned

The goal of this project was for me to deepen my understanding of fundamental networking concepts through hands-on implementation. I had a basic understanding of how network traffic flows across the Internet. Building this tool helped develop my understanding through practicality. This project helped bridge the gap between what I understood from textbooks and real-world network traffic.

8.1 From theory to real-world networking / Real-world ICMP limitations

The unpredictability and inconsistency of real network behavior was eye-opening for me. One of the first things I realized when working with ICMP is that not receiving responses from routers

or endpoints doesn't necessarily mean there is network failure. While routers use TTL to decide whether to forward or drop a packet, I also discovered that many networks intentionally filter or deprioritize ICMP messages as a network security practice. As a result, some hops may not respond to probes even when traffic is successfully passing through them. This can give the impression that a network is experiencing congestion, packet loss, or connectivity problems even when traffic is being forwarded normally. This taught me that interpreting traceroutes results requires understanding real-world behavior rather than relying on raw probe behavior.

8.2 Low-level networking challenges

Developing this program has introduced me to the complexities of low-level networking. Normal application programming is more abstract than low-level networking. This requires an awareness of how the operating system and network stack behave. Working with raw sockets on macOS also introduced the need for elevated permissions due to certain restrictions. Understanding packet construction, ICMP message handling, and packet headers required me to think more carefully about protocols and system behavior during development.

8.3 Network performance is multi-dimensional

Good network analysis requires identifying the performance metrics that best indicate the type of network behavior being evaluated and interpreting those metrics to provide an accurate picture of overall network performance. Measuring latency, packet loss, or jitter in isolation could give you misleading results negatively affecting bottleneck analysis. For example, latency alone may appear acceptable while packet loss or jitter indicates instability. Using multiple metrics gives a more comprehensive picture of overall network health and performance.

I also learned that the interpretation portion of the analysis is just as important as calculation. Choosing thresholds for high RTT, jitter, packet loss, etc. is not an exact science. There is no universal benchmark that applies to every network. Using thresholds that are too stringent or lenient can produce a misleading bottleneck scoring system and inaccurate performance analysis. Network analysis is not only about collecting raw measurements but applying interpretations that are meaningful in context.

9. Future Improvements

9.1 Long-Term Network Analysis

Future iterations of this project could include additional utilities for long-term network analysis. The purpose of the export module was to provide a foundation for these future utilities allowing performance data to be stored for later use. Long-term monitoring and historical route analysis assist with defining a network baseline for latency, packet loss, jitter, and bandwidth across network segments. Once you have a baseline, anomaly detection becomes meaningful, allowing performance degradation or unusual network behavior to be tracked over time.

9.2 Enhanced Reporting Capabilities

Enhanced reporting capabilities could also be implemented to enhance usability; graphical visualizations of network performance metrics would make trends and anomalies easier to identify.

9.3 IPv6 Support

Additionally, most of my current implementation is focused on IPv4. Expanding the toolkit to support IPv6 would improve compatibility with modern network environments and increase the tool's real-world usability. Collectively, these improvements would increase the toolkit's practicality for both educational and real-world network diagnostics.

10. Conclusion

Through the development of an enhanced traceroute and network diagnostic toolkit I've explored foundational networking concepts such as how network traffic flows across the internet, routing and packet forwarding, and hops and path discovery. I've also gained hands-on experience with Go/systems programming as well as network performance analysis, which is something I value given my career goals. I successfully implemented a traceroute module that maps the route a packet takes to an endpoint along with supporting modules that allow you to compare this route with another route that has an option for simulated delay and packet loss. I designed an analysis framework that computes metrics and reports hops statistics, calculating a bottleneck score based on these statistics. I also implemented a ping module that performs ICMP echo requests and a module that measures the time it takes to establish application-level connectivity. The last module I implemented allows for the results from the traceroute, ping, and TCP to be exported into JSON files for long-term analysis. One of the most significant accomplishments of this project was successfully implementing a traceroute utility using raw ICMP sockets. Developing this module required a much deeper understanding of ICMP messaging, TTL manipulation, packet parsing, and low-level network programming than I had prior to beginning the project. Another key accomplishment in this project was the design of a modular codebase that separates the collection, analysis, reporting, and export functionality. This architecture improves

maintainability, reusability, and scalability while making future expansion and debugging much more efficient. The implementation of a weighted bottleneck detection system was another key accomplishment. Using multiple performance metrics to give a comprehensive analysis report is a core concept in my analysis of network paths. Also, the ability to serialize and save traceroute, ping, and TCP latency results to perform long-term analysis through JSON export is another key accomplishment of this project. This project has strengthened my understanding of network engineering and systems programming. I've become familiar with ICMP packet construction and working with raw sockets. Overall, this project allowed me to apply networking theory to a practical implementation, giving me a deeper understanding of how modern network diagnostic tools operate while providing a strong foundation for future work in networking and cybersecurity.

References

- Cloudflare. (n.d.). *Time to live (TTL)*. <https://www.cloudflare.com/learning/cdn/glossary/time-to-live-ttl/>
- King, G. (2014, March 1). *Raw sockets in Go: IP layer*. Darkcoding. <https://darkcoding.net/uncategorized/raw-sockets-in-go-ip-layer/>
- KMCD. (2025, December 9). *Traceroute tool from scratch in Go*. <https://kmcd.dev/posts/traceroute/>
- NetBeez. (n.d.). *Network performance analysis*. <https://netbeez.net/blog/network-performance-analysis/>
- Peterson, L., & Davie, B. (n.d.). *An Introduction to Computer Networks* (2nd ed.). University of Minnesota Open Textbook Library. <https://open.umn.edu/opentextbooks/textbooks/353>
- Postel, J. (1981). *Internet Control Message Protocol (RFC 792)*. RFC Editor. <https://www.rfc-editor.org/rfc/rfc792>
- Postel, J. (1981). *Internet Protocol (RFC 791)*. RFC Editor. <https://www.rfc-editor.org/rfc/rfc791>
- The Go Authors. (n.d.). *Package net*. Go Packages. <https://pkg.go.dev/net>
- The Go Authors. (n.d.). *Package icmp*. Go Packages. <https://pkg.go.dev/golang.org/x/net/icmp>
- The Go Authors. (n.d.). *Package ipv4*. Go Packages. <https://pkg.go.dev/golang.org/x/net/ipv4>
- Wireshark Foundation. (n.d.). *Wireshark User's Guide*. https://www.wireshark.org/docs/wsug_html_chunked/